

Understanding Flash Attention in GPT-2 Training

Pranavi Pathakota
Computer Science
Texas A&M University
pranavi.pathakota@tamu.edu

Abstract—Training large language models (LLMs) faces memory bottlenecks because of the quadratic complexity of standard attention mechanism. I have trained LLMs using Flash Attention, a memory-efficient attention algorithm using cuDNN 9.0+, integrated into the llm.c <https://github.com/karpathy/llm.c.git> training framework for GPT-2 models. Through extensive experiments on NVIDIA A100 GPUs (40GB), I evaluated three model sizes (124M, 350M, and 774M parameters) across varying sequence lengths (1024, 2048, 4096 and 8192 tokens) in both single-GPU and multi-GPU (4× data parallel) configurations. I have also performed a single experiment in multi-node settings (4 nodes with 4 gpus each), so in total (16x data parallel). The results demonstrate that Flash Attention achieves 41% memory reduction for 124M models and enables 2× longer sequence training compared to standard attention. The same is observed in case of 350M model. And for the 774M model, 4x multi-GPU training with Flash Attention achieves 142K tokens/sec throughput with 56.9% model FLOPs utilization (MFU) at sequence length 1024, representing a 35% improvement over standard attention (106K tokens/sec, 42.5% MFU). I have also analyzed scaling efficiency, memory characteristics, and the advantage of using Flash Attention in training Large Language Models.

Index Terms—Flash Attention, Large Language Models, GPU Acceleration, Memory Optimization, Distributed Training, cuDNN

I. INTRODUCTION

A. Background and Motivation

The transformer architecture has revolutionized natural language processing, enabling exceptional capabilities in large language models (LLMs) such as GPT-3, GPT-4, and LLaMA. For modern LLMs, the context length has been increased to even million tokens. For the applications such as RAG, knowledge sources, it is very important that we enable training the LLMs for this large sequences. However, training these models faces significant computational challenges, particularly in the attention mechanism, which exhibits $O(N^2)$ time and space complexity with respect to sequence length N .

For LLMs processing long contexts (4K-32K tokens) in resource constraint settings, the standard attention mechanism becomes a critical bottleneck:

- Memory Storage: Activations scale quadratically, exceeding GPU memory capacity
- Bandwidth bottleneck: Repeated HBM (High Bandwidth Memory) reads/writes dominate runtime
- Scalability limits: Longer sequences essential for understanding context become impractical

B. Problem Statement

I have performed various series of experiments mainly testing for the scalability and multi gpu settings.

This project addresses three key questions:

- 1) Can Flash Attention reduce memory sufficiently to enable training larger models and longer sequences effectively using GPU hardware?
- 2) How Flash Attention influences the memory savings and computational throughput?
- 3) How does Flash Attention scale in multi-GPU data parallel training scenarios?

C. Contributions

My contributions include:

- Integration of cuDNN-based Flash Attention into the llm.c training framework
- Evaluation across 3 model sizes, 4 sequence lengths, 2 attention variants, utilizing data parallel.
- Considered single-GPU vs. 4-GPU data parallel performance

II. RELATED WORK

A. Attention Mechanisms

Vaswani et al. (2017) [1] introduced the transformer architecture with scaled dot-product attention. While effective, this approach stores the full attention matrix ($N \times N$), causing memory bottlenecks for long sequences.

Various approaches have attempted to reduce attention complexity. Some of the methods include sparse attention mechanism [2], Linear attention approximations [3], and kernel fusion and recomputation strategies

B. Flash Attention

Dao et al in 2022 introduced FlashAttention [4], an IO-aware attention algorithm that fuses attention operations to minimize HBM accesses. It has also used tiling and recomputation to avoid storing the full attention matrix in HBM and instead computed the block wise attention in SRAM which is faster memory access compared to HBM. The authors have shown in the paper that this method achieves exact attention (no approximation) with 2-4× speedup.

Dao improved over the proposed method [4] and introduced FlashAttention-2 [5] which further optimizations including, better parallelization over sequence length and work partitioning improvements.

cuDNN Flash Attention (NVIDIA, 2023): NVIDIA integrated Flash Attention into cuDNN 9.0+, a deep learning library providing hardware-optimized kernels for A100 GPU configurations, this can be directly integrated in training LLMs.

C. Distributed Training Frameworks

1) *Data Parallelism*:: Standard approach where each GPU processes different data batches and in the code, I have employed ZeRO-1 optimizer state sharding [6] for memory efficiency.

2) *LLM Training Frameworks*:: Some of the training frameworks which can be implemented with minimal changes are as follows:

- *llm.c* : Minimalist training framework in C/CUDA
- Megatron-LM : Production-scale training with model parallelism

For the following experiments, I have used *llm.c* to test the advantages of flash attention for training of LLMs.

III. METHODOLOGY

A. Flash Attention Algorithm

Flash Attention reformulates the attention computation to minimize memory reads/writes

Standard Attention:

$$\begin{aligned} Q, K, V &\in \mathbb{R}^{N \times d} \\ S &= QK^\top \in \mathbb{R}^{N \times N} \quad \# O(N^2) \text{ memory} \\ P &= \text{softmax}(S) \in \mathbb{R}^{N \times N} \quad \# O(N^2) \text{ memory} \\ O &= PV \in \mathbb{R}^{N \times d} \end{aligned}$$

Algorithm 1 Flash Attention

Input: $Q, K, V \in \mathbb{R}^{N \times d}$, block sizes B_r, B_c

Output: $O \in \mathbb{R}^{N \times d}$

Divide Q into blocks $Q_1, \dots, Q_{T_r}$ of size B_r . Divide K, V into blocks $K_1, \dots, K_{T_c}$ and $V_1, \dots, V_{T_c}$ of size B_c . Initialize $O = 0$, $\ell = 0$, $m = -\infty$

```

for  $i = 1$  to  $T_r$  do
  Load  $Q_i$  from HBM to SRAM
  for  $j = 1$  to  $T_c$  do
    Load  $K_j, V_j$  from HBM to SRAM
    Compute  $S_{ij} = Q_i K_j^\top$  on SRAM
    Compute  $\tilde{m}_{ij} = \text{rowmax}(S_{ij})$ 
    Compute  $\tilde{P}_{ij} = \exp(S_{ij} - \tilde{m}_{ij})$ 
    Compute  $\tilde{\ell}_{ij} = \text{rowsum}(\tilde{P}_{ij})$ 
    Update  $m, \ell, O$  (online softmax)
  Write  $O_i$  back to HBM

```

return O

IV. EXPERIMENTAL SETUP

A. Hardware Platform

System: NERSC Perlmutter Supercomputer

- GPU: NVIDIA A100-SXM4-40GB
- Interconnect: NVIDIA NVLink

B. Software Environment

CUDA Stack:

- CUDA: 12.4
- cuDNN: 9.0+ (with Flash Attention support)
- NCCL: 2.24.3 (multi-GPU communication)

C. Model Configurations

Below table shows different model configurations I have tested for the experiments.

Model	Parameters	Layers	Hidden Dim	Heads
GPT-2 124M	124M	12	768	12
GPT-2 350M	350M	24	1024	16
GPT-2 774M	774M	36	1280	20

TABLE I
MODEL CONFIGURATIONS

Sequence Lengths Tested: $T \in \{1024, 2048, 4096, 8192\}$ tokens

D. Training Hyperparameters

a) *Optimizer*:: I used the AdamW optimizer with the following settings:

- learning rate: 6×10^{-4}
- Learning rate schedule: cosine decay
- Weight decay: 0.1
- $\beta_1 = 0.9$, $\beta_2 = 0.95$
- Gradient clipping threshold: 1.0

b) *Batch Configuration*::

- Target total batch size: 524,288 tokens
- Single GPU: $B = 32$ with gradient accumulation as required, which is computed according to set target batch
- Multi-GPU 4x: $B = 8$ per GPU with gradient accumulation

c) *Training Duration*::

- One-hour training runs for benchmarking
- Approximately 200–1000 optimization steps, depending on configuration

E. Dataset

Fineweb10B is a 10-billion-token dataset derived from filtered Common Crawl and tokenized using the GPT-2 BPE tokenizer (vocab size 50,257)

V. RESULTS

This section mainly discuss about the advantages of integrating flash attention w.r.t memory, throughput and model flop utilization (mfu). I have also explained in-detailed results for Single GPU vs. Multi-GPU settings

1) *Memory Comparison: Single GPU (B=32, T=1024):*

Configuration	Memory (GB)	Reduction vs. Standard
Standard Attention	22.0	–
Flash Attention	13.0	41% ↓

TABLE II
SINGLE-GPU 124M MEMORY USAGE

Multi-GPU (4× GPUs, B=8 per GPU, T=1024):

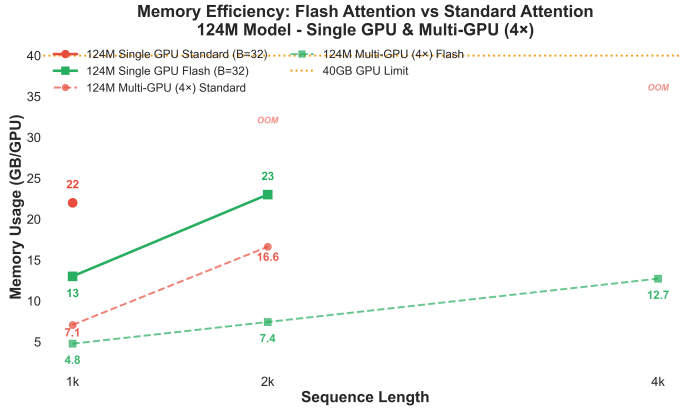


Fig. 1. Memory Usage

Configuration	Memory/GPU (GB)	Reduction vs. Single GPU
Standard	7.06	68% ↓
Flash	4.76	63% ↓

TABLE III
MULTI-GPU 124M MEMORY USAGE

Memory Scaling with Sequence Length: 124M Model, Flash Attention, Multi-GPU:

Sequence Length	Memory/GPU (GB)	Scaling Factor
T=1024	4.76	1.0×
T=2048	7.42	1.56×
T=4096	12.72	2.67×
T=8192	OOM	—

TABLE IV
MEMORY SCALING WITH SEQUENCE LENGTH 124M MODEL

Figure 1 shows these results and it can be concluded that Flash Attention enables fitting larger models in memory, but multi-GPU provides even greater memory efficiency through data parallelism.

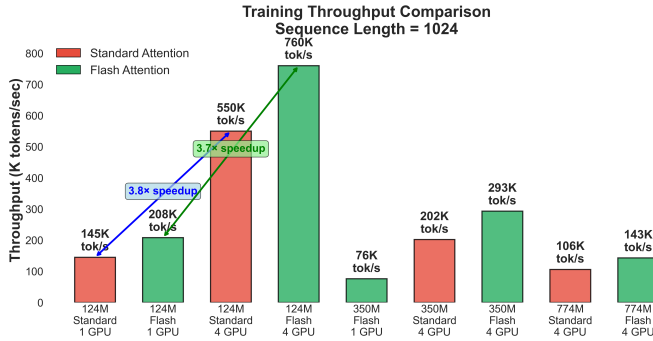


Fig. 2. Throughput (Tok/s) for T = 1024

2) *Throughput Comparison*: Figure 2 shows that enabling Flash Attention provides 38-44% throughput improvement across all configurations. From scaling Single GPU to 4x Multi-GPU, efficiency (tok/sec) for 124M model is 3.8x which

is theoretically expected result. Standard Attention cannot fit 350M on single 40GB GPU at T=1024. Multi-GPU is essential for 350M training to enable Standard Attention. 774M demonstrates absolute need for multi-GPU with 40GB A100s. Single GPU training even with Flash Attention enabled is infeasible on standard hardware for 774M model.

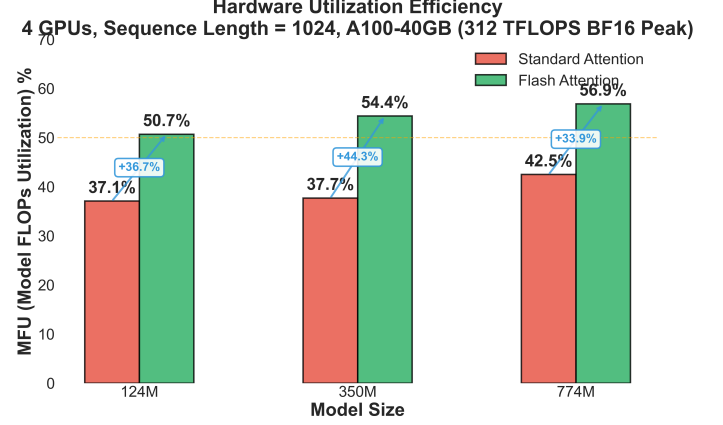


Fig. 3. MFU across varying model sizes and T = 1024

3) *Model Flops Utilization*: Figure 3 shows Multi-GPU settings across different model sizes where Flash Attention increases model FLOPs utilization from 37-42% to 51-57%, indicating better hardware utilization. The highest utilization is observed for 774M model.

A. Single GPU Experiments

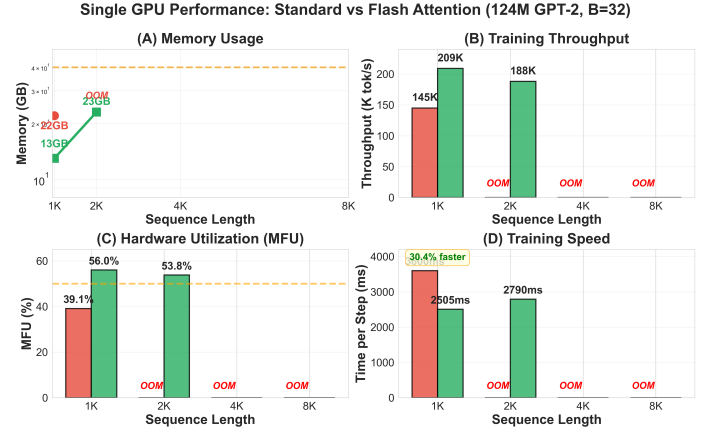


Fig. 4. Single GPU: 124M model

From figure 4, for T = 1024, memory requirement has been reduced from 22GB to 13GB by using Flash Attention, which is 41% reduction and throughput has increased by 44.2% (209K vs 145K tokens/sec at T = 1024). For T = 2048, Standard Attention is not able to fit in the memory where as Flash Attention is able to run. Flash Attention has enabled higher utilization of MFU where as this almost stays constant across

the context lengths, and a small dip is observed as context length is increased to $T = 2048$, this may be attributed to the memory bound for higher data access.

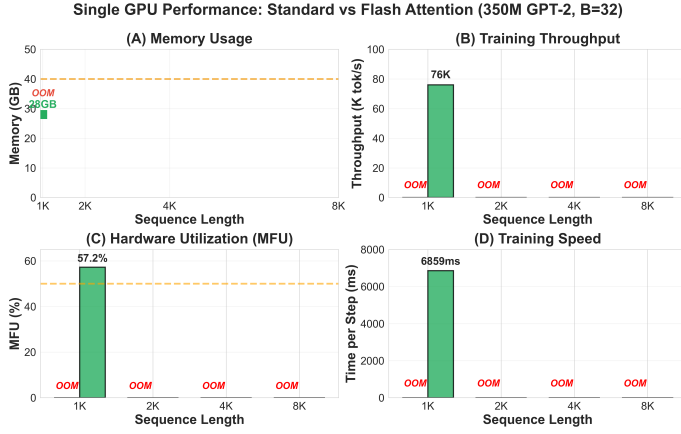


Fig. 5. Single GPU: 350M model

From figure 5, for $T = 1024$, Standard Attention is unable to run where as Flash Attention enables this. 350M model is unable to run for any other contexts lengths even when Flash Attention is enabled.

For Single GPU settings, 774M, I was unable to run for any context lengths, even with Flash Attention enabled.

B. Multi GPU Experiments

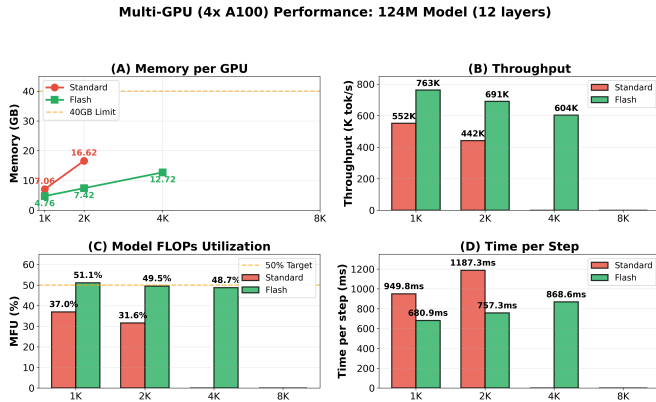


Fig. 6. Multi-GPU: 124M model

From figure 6, Flash Attention on 4x A100 GPUs achieves 38-56% higher throughput (763K vs 552K tok/s at $T=1024$, 691K vs 442K tok/s at $T = 2048$) and 33-55% memory reduction (4.76GB vs 7.06GB per GPU at $T = 1024$, 7.42GB vs 16.62GB per GPU at $T = 2048$) compared to Standard Attention for GPT-2 124M. It enables 2x longer sequences ($T = 4096$ vs $T = 2048$) within 40GB limits, though MFU decreases with longer sequences (51.1% to 48.7%) due to memory bandwidth bottlenecks.

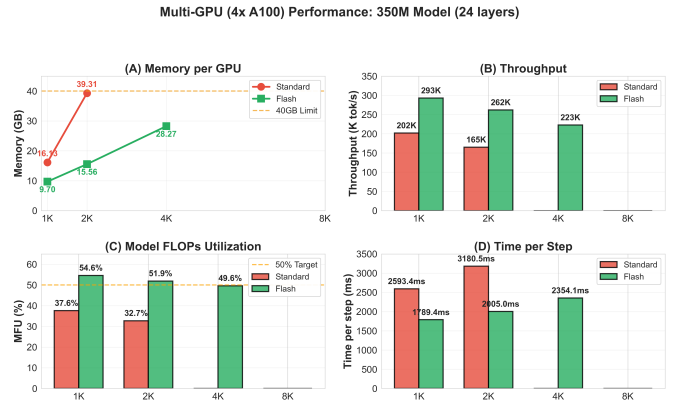


Fig. 7. Multi-GPU: 350M model

From figure 7, Flash Attention achieves 45-59% higher throughput (293K vs 202K tok/s at $T = 1024$, 262K vs 165K tok/s at $T = 2048$) with 40-60% memory reduction (9.70GB vs 16.13GB per GPU at $T = 1024$, 15.56GB vs 39.31GB per GPU at $T = 2048$). It enables training at $T=4096$ (223K tokens/sec, 28.27GB) where Standard Attention exceeds 40GB limits, though MFU decreases from 54.6% to 49.6% with longer sequences.



Fig. 8. Multi-GPU: 774M model

From figure 8, Flash Attention provides 34% throughput gain (142K vs 106K tokens/sec at $T = 1024$) with 42% less memory (16.93GB vs 28.99GB per GPU at $T = 1024$) and achieves highest MFU (56.9% vs 42.5%). Enables $T=2048$ training (131K tokens/sec, 27.08GB) where Standard Attention fails due to memory constraints

Multi Node Multi GPU: I have performed a single experiment considering model size of 774M, where I have tried scaling up the current settings with 4 Nodes with 4 GPUs each, so 16x GPU in total. The slurm script required a lot of changes to ensure it recognizes all the GPUs across the nodes and assigns the correct rank across each of them.

16-GPU Performance: 774M Model (36 layers)

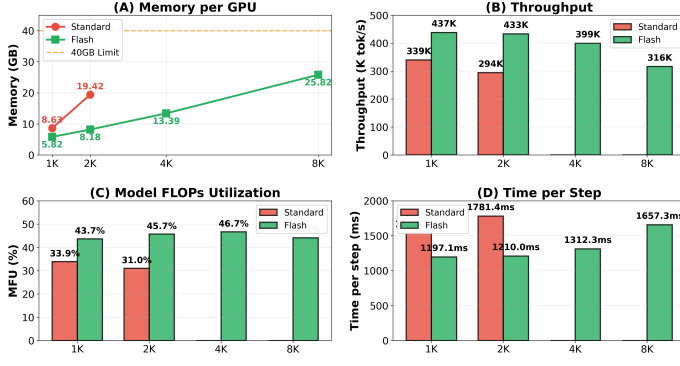


Fig. 9. Multi-Node Multi-GPU (16x A100): 774M model

From figure 9, scaling the 774M model to 16 GPUs across 4 nodes reveals Flash Attention’s essential role in multi-node training: it provides 28-47% throughput improvements (340K to 438K tok/s at $T = 1024$, 294K to 433K tok/s at $T = 2048$) while reducing memory by 33-58% per GPU. Standard attention fails beyond 2K sequences even with 16-way parallelism (OOM at $T = 4096 / 8192$), while Flash Attention successfully trains up to $T = 8192$ sequences (316K tok/s, 25.8 GB/GPU, 44.1% MFU). At $T = 1024$, for 4x GPU settings with Flash Attention enabled, throughput is 142 tok/s, whereas for 16x GPU settings, throughput is 437 tok/s. Therefore, 77% scaling efficiency ($437/142 \approx 3.08\times$ speedup from 4 GPUs) demonstrates efficient communication across nodes and Flash Attention enables large model training on standard 40GB hardware.

It can also be speculated that enabling Flash attention can result in the training of all the model sizes across all the varying context lengths since I am able to train for the larger size of 774M model.

VI. CONCLUSION

The key takeaways from this project are that Flash Attention provides excellent throughput improvements while enabling training of significantly longer sequences that would otherwise run out of memory. Flash Attention achieves higher Model FLOPs Utilization (MFU) compared to standard attention, demonstrating better hardware efficiency by reducing memory bandwidth bottlenecks. For multi-GPU settings, I have observed strong scaling efficiency on 4 GPUs with near-linear speedup, though efficiency decreases to acceptable levels on 16 GPUs due to multi-node communication overhead. Most critically, Flash Attention enables training larger models on standard 40GB GPUs with long context lengths, while standard attention fails at these configurations even with multi-GPU parallelism. The main technical challenges included integrating cuDNN Flash Attention into the llm.c framework and configuring multi-node SLURM scripts for optimal A100 GPU utilization.

REFERENCES

- [1] "Attention Is All You Need", Ashish Vaswani and Noam Shazeer and Niki Parmar and Jakob Uszkoreit and Llion Jones and Aidan N. Gomez and Lukasz Kaiser and Illia Polosukhin
- [2] Sparsers is Faster and Less is More: Efficient Sparse Attention for Long-Range Transformers , Chao Lou and Zixia Jia and Zilong Zheng and Kewei Tu, 2024
- [3] Linear Self-attention Approximation via Trainable Feedforward Kernel, Springer Nature Switzerland, Yorsh, Uladzislau and Kovalenko, Alexander
- [4] FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness, Tri Dao and Daniel Y. Fu and Stefano Ermon and Atri Rudra and Christopher Ré.
- [5] FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning, Tri Dao.
- [6] ZeRO: Memory Optimization Towards Training A Trillion Parameter Models, Samyam Rajbhandari and Jeff Rasley and Olatunji Ruwase and Yuxiong He.